AI Copilot Code Review Using Large Language Models

Bandi Srihitha

(Koneru Lakshmaiah Education Foundation, Hyderabad, Telangana, India
ORCID: [optional], [2420030099@klh.edu.in])

Prathipati Pranathi

(Koneru Lakshmaiah Education Foundation, Hyderabad, Telangana, India
ORCID: [optional], [2420030043@klh.edu.in])

Emani Mahathi

(Koneru Lakshmaiah Education Foundation, Hyderabad, Telangana, India
ORCID: [optional], [2420030093@klh.edu.in])

Abstract: Software code review is a fundamental software-engineering activity for detecting defects, security concerns, maintainability problems, and violations of coding standards. Manual review, however, is time-consuming and dependent on reviewer expertise. Traditional static analyzers provide deterministic findings but have limited semantic and repository-level awareness, whereas Large Language Models (LLMs) can explain source code and generate review suggestions but may produce generic, inconsistent, or unsupported comments when relevant context is unavailable. This paper presents an AI Copilot for context-aware automated code review that combines language-specific static analysis, CodeBERT semantic retrieval, FAISS vector search, Retrieval-Augmented Generation (RAG), and the code-specialized CodeLlama model. The proposed framework supports Python, Java, and JavaScript using Pylint, PMD, and ESLint, respectively. Source-code representations are embedded using CodeBERT and indexed in FAISS so that semantically similar code can be retrieved from CodeSearchNet and selected public GitHub repositories. Static findings, retrieved evidence, source code, and review instructions are assembled into a structured prompt for CodeLlama, which produces an explainable review containing issue descriptions, severity, evidence, suggested fixes, and confidence information. Prototype evaluation reports overall precision of 0.56, recall of 1.00, F1-score of 0.71, and accuracy of 0.56 across the evaluated multilingual static-analysis set. Retrieval evaluation reports Recall@1 of 0.67, Recall@2 of 0.67, Recall@3 of 1.00, and mean reciprocal rank of 0.78. Review-quality evaluation reports precision of 1.00, recall of 0.67, F1-score of 0.80, severity accuracy of 1.00, evidence-grounding rate of 1.00, and unsupported-finding rate of 0.00. These results demonstrate the feasibility of integrating deterministic analysis, semantic retrieval, and LLM reasoning into a unified explainable code-review workflow, while also highlighting the need for larger matched benchmark evaluations.

Key Words: automated code review, large language models, CodeLlama, CodeBERT, FAISS, retrieval-augmented generation, static analysis, explainable AI, software engineering

Category: D.2.5, D.2.9, H.3.3, I.2.7

DOI:

1 Introduction

Software systems are increasingly developed through rapid, collaborative workflows in which large numbers of source-code changes must be inspected before integration. Code review is therefore a critical software-engineering practice for identifying defects, security weaknesses, maintainability problems, and violations of established development practices. At the same time, manual review requires substantial developer effort and depends on the availability of reviewers with appropriate expertise. The increasing scale of repositories and frequency of code changes make automated assistance attractive for both individual developers and development teams.

Traditional static-analysis tools provide an established approach to automated review. They apply deterministic rules to source code and can efficiently identify syntax problems, coding-standard violations, suspicious constructs, and selected classes of defects. Their principal limitation is that the findings are generally produced from local program structure and predefined rules. They do not inherently reason over the broader repository history, related implementations, developer intent, or semantically similar code.

Large Language Models (LLMs) provide a complementary capability. Models trained on programming languages can summarize code, explain implementation decisions, identify potential issues, and propose modifications. Code-specialized models such as CodeLlama were introduced specifically to support programming-related tasks [Rozière et al. 2023]. However, LLM-based review can be sensitive to prompt quality and available context. A model may generate a plausible-sounding comment without sufficient evidence, overlook repository-specific conventions, or produce inconsistent severity labels. These limitations motivate systems that combine deterministic evidence with semantic reasoning rather than treating an LLM as the only reviewer.

Recent research has explored several directions. AI-powered code-review agents have investigated the use of LLMs for detecting code smells, potential bugs, and improvement opportunities [Rasheed et al. 2024]. CodeAgent uses multiple communicating agents to emulate aspects of collaborative review [Tang et al. 2024]. RepoAudit combines LLM-driven repository exploration with validation mechanisms to reduce hallucinated findings [Guo et al. 2025]. Fine-tuned LLM approaches have targeted both review accuracy and comprehensibility [Yu et al. 2025]. Retrieval-based approaches have also gained attention because relevant historical reviews and contextual evidence can improve generation quality [Meng et al. 2025], [Zhang et al. 2026].

The proposed work follows a hybrid approach. Instead of choosing between static analysis and LLM reasoning, the framework uses both. Pylint, PMD, and ESLint produce language-specific findings; CodeBERT [Feng et al. 2020] provides semantic representations of source code; FAISS [Johnson et al. 2021] enables efficient similarity search; Retrieval-Augmented Generation (RAG) [Lewis et al. 2020] supplies relevant retrieved context; and CodeLlama generates the final contextual review.

The main contributions of this work are:

- *Unified multilingual pipeline:* a single code-review pipeline for Python, Java, and

JavaScript.

- *Hybrid evidence:* integration of deterministic static-analysis findings with LLM-generated explanations.
- *Repository-aware retrieval:* CodeBERT and FAISS based semantic retrieval for repository-aware context.
- *RAG prompt design:* a prompt that combines source code, static findings, retrieved context, and review instructions.
- *Explainable output:* structured review output containing severity, evidence, suggested fixes, and confidence.
- *Evaluation:* an experimental evaluation covering language-wise static analysis, semantic retrieval, and review quality.

The remainder of this paper is organized as follows. [Section 2] reviews related work. [Section 3] presents the problem statement, objectives, and research gap. [Section 4] describes the proposed architecture and methodology. [Section 5] covers the datasets, technology stack, experimental setup, evaluation metrics, results, comparative analysis, discussion, and limitations. [Section 6] concludes the paper.

2 Related Work

2.1 LLM-Based Code Review

The use of LLMs for code review has developed rapidly because these models can process both programming constructs and natural-language instructions. Rasheed et al. investigated an LLM-based AI agent for code review and discussed its ability to identify code smells, potential bugs, and improvement opportunities [Rasheed et al. 2024]. Such systems offer broader semantic reasoning than conventional rule-based analyzers, but their outputs may still require validation.

CodeAgent introduced a multi-agent architecture for automated code review. It uses a supervisory QA-checker to ensure that agent contributions address the original review question and evaluates tasks including vulnerability introduction, style adherence, inconsistency detection, and code revision [Tang et al. 2024]. The approach illustrates how decomposition and validation can improve automated review, but multi-agent coordination also introduces additional workflow complexity.

Repository-level auditing introduces a different challenge because an LLM must reason over a large and potentially interconnected codebase. RepoAudit addresses context limitations by allowing an autonomous agent to explore a repository and incorporates validation to reduce false positives [Guo et al. 2025]. This motivates the present framework's emphasis on retrieval of relevant code context before generating a review.

2.2 Pretrained Code Models

CodeBERT is a pretrained model designed to learn representations of programming and natural languages [Feng et al. 2020]. Its embeddings can be used to represent code semantically and support similarity-based retrieval. CodeXGLUE provides a benchmark collection for code understanding and generation tasks [Lu et al. 2021], while CodeSearchNet provides resources for semantic code search [Husain et al. 2019].

CodeLlama provides code-specialized foundation models for code generation and understanding [Rozière et al. 2023]. In the proposed framework, CodeLlama is not used in isolation. Its generation stage receives evidence from static analysis and semantic retrieval, allowing the LLM to operate on a more structured review context.

2.3 Retrieval-Augmented Code Review

Retrieval-Augmented Generation (RAG) was introduced as a method for combining parametric generation with non-parametric external memory [Lewis et al. 2020]. In software-engineering applications, retrieval can provide historical reviews, related implementations, or repository-specific evidence.

RARe, described as Retrieval-Augmented Reviewer, combines retrieval and generative review by selecting relevant reviews from a codebase and supplying them to a neural generator [Meng et al. 2025]. LAURA further emphasizes review-knowledge retrieval, context augmentation, and systematic guidance for LLM-based review generation [Zhang et al. 2026]. These approaches support the idea that relevant contextual information can improve the usefulness of automatically generated review comments.

2.4 Hybrid Static Analysis and LLM Reasoning

Jaoua et al. proposed combining large language models with static analyzers through knowledge integration at data preparation, inference, and post-inference stages [Jaoua et al. 2025]. Their work explicitly motivates hybrid approaches that combine the precision of rule-based systems with the broader language understanding of LLMs.

The present framework adopts the same general principle but organizes it as an end-to-end application in which static-analysis findings, semantic retrieval, and RAG are integrated before CodeLlama generation. The final output is also structured to include severity and supporting evidence.

2.5 Review Comprehension and Explainability

Automated review quality is not determined only by whether a system identifies an issue. Developers must also understand why an issue matters and how it can be corrected. Yu et al. studied fine-tuning approaches aimed at improving both accuracy and comprehensibility of automated code review [Yu et al. 2025]. CodeReviewQA provides an assessment framework for code-review comprehension in LLMs [Lin et al. 2025]. These studies reinforce the importance of understandable review comments and motivate the structured output used in the proposed system.

3 Problem Statement, Objectives, and Research Gap

3.1 Problem Statement

Software code review is essential for maintaining code quality, but manual inspection is time-consuming and depends on reviewer expertise. Static analyzers reduce this burden by providing deterministic findings, yet they lack broad semantic and repository context. LLMs can provide richer explanations but may produce unsupported or inconsistent recommendations when they are not grounded in relevant code context. Therefore, there is a need for a hybrid, context-aware automated code-review framework that combines deterministic analysis, semantic retrieval, retrieval-augmented prompting, and code-specialized LLM reasoning.

3.2 Objectives

The project objectives are to:

1. develop multi-language code analysis for Python, Java, and JavaScript;
2. integrate language-specific static analyzers to detect deterministic rule-based issues;
3. generate semantic code embeddings using CodeBERT;
4. build a FAISS-based vector database for efficient similar-code search;
5. implement a RAG pipeline using CodeSearchNet and selected GitHub repositories;
6. construct structured prompts containing source code, static findings, retrieved context, and review instructions;
7. use CodeLlama to generate contextual reviews and suggested fixes;
8. provide explainable severity-ranked recommendations with supporting evidence and confidence information; and
9. evaluate retrieval and review performance using appropriate benchmark and repository data.

3.3 Research Gap

Several limitations can be observed in existing approaches. First, existing systems can have limited repository or contextual awareness. Second, static analysis and LLM review are often treated as separate stages rather than complementary evidence sources. Third, generated comments can be difficult to validate. Fourth, severity labels can vary between generated outputs. Finally, many approaches do not provide a unified multilingual review interface. [Tab. 1] summarizes each gap and the corresponding response in the proposed framework.

Observed gap	Response in proposed framework
Limited repository context	CodeBERT embeddings and FAISS retrieval
Static and LLM review separated	Static findings are included in the LLM prompt
Generated comments difficult to validate	Structured output with evidence and confidence
Inconsistent severity labels	Deterministic severity normalization
Limited language coverage	Python, Java, and JavaScript analyzers
Inconsistent result presentation	Streamlit structured review report

Table 1: Research gaps and proposed responses.

3.4 Innovation, Novelty, and Expected Outcomes

The innovation of the proposed framework is primarily architectural. It combines repository-aware FAISS-based RAG, CodeLlama, static analysis, intelligent retrieval of similar code, explainable suggestions, and severity-prioritized reporting in one application. The documented novelty includes CodeBERT-based semantic code retrieval, repository-aware contextual generation, multi-language support, and structured review output. Expected outcomes include:

- automated review for Python, Java, and JavaScript;
- improved context-aware code analysis;
- explainable review suggestions;
- severity-based prioritization;
- reduced manual review effort;
- improved developer productivity and maintainability; and
- a scalable basis for future programming-language support.

4 Proposed Methodology

4.1 Overall Architecture

The proposed framework is a hybrid context-aware code-review system. [Fig. 1] illustrates the complete architecture. The first stage receives source code and identifies the programming language. Language-specific analyzers then generate deterministic findings. In parallel, source-code representations are embedded using CodeBERT and used to retrieve similar code from the indexed corpus.

The retrieved evidence and static-analysis findings are merged with the original source code and review instructions. This contextual prompt is sent to CodeLlama. The final response is parsed and presented as an explainable report containing the identified issue, explanation, severity, suggested fix, supporting evidence, and confidence estimate.

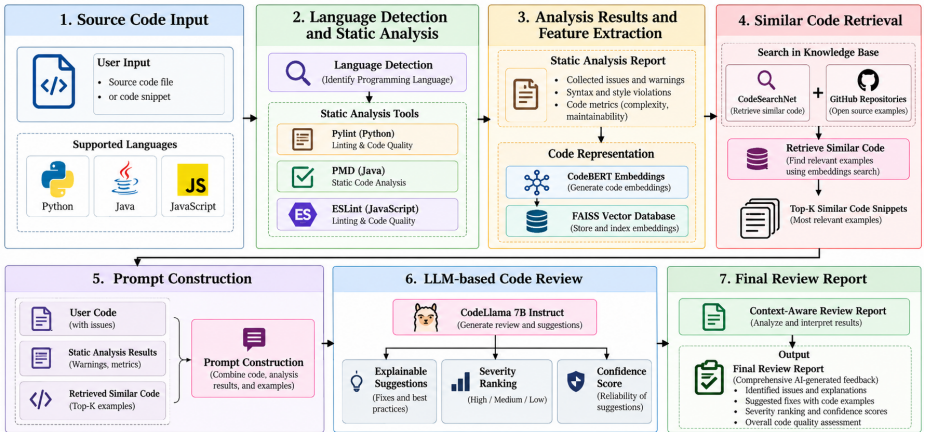


Figure 1: Proposed hybrid context-aware architecture for AI-assisted code review.

4.2 End-to-End Workflow

The implementation follows eight stages, shown in [Fig. 2].

1. *Source-code upload:* the user submits the code to the application.
2. *Language detection:* the application determines whether the input is Python, Java, or JavaScript.
3. *Static analysis:* the appropriate analyzer generates deterministic findings.
4. *Embedding generation:* CodeBERT converts code into semantic representations.
5. *Similarity retrieval:* FAISS searches the indexed code representations.
6. *Contextual prompt construction:* source code, static findings, retrieved context, and review instructions are combined.

7. *Review generation:* CodeLlama produces the context-aware review.
8. *Explainable reporting:* the generated information is structured and displayed through the Streamlit interface.

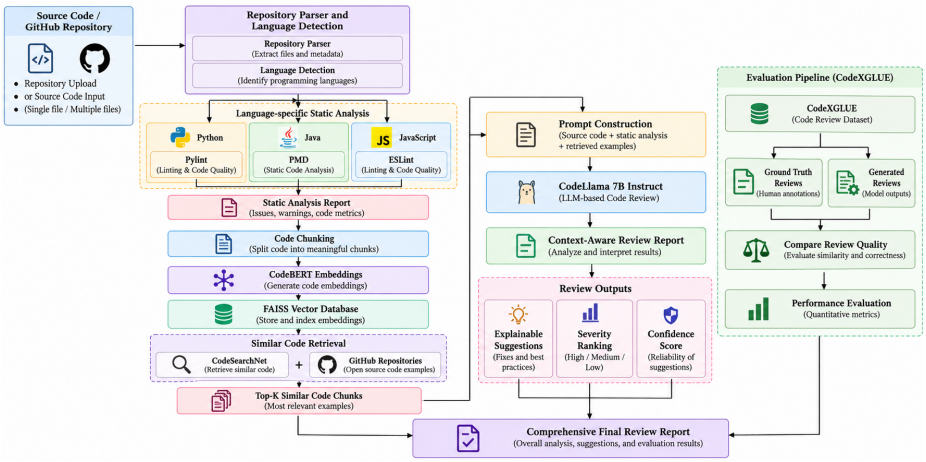


Figure 2: Eight-stage workflow of the proposed AI Copilot code-review framework.

4.3 Static-Analysis Layer

The static-analysis layer is responsible for deterministic and language-specific inspection. Python source code is routed to Pylint, Java source code to PMD, and JavaScript source code to ESLint. The analyzers provide rule-based findings that form an explicit evidence layer for subsequent reasoning.

This stage is important because an LLM-only architecture may generate plausible but unsupported comments. Static findings provide concrete signals that can be retained and referenced in the generated review. The design therefore treats static analysis as an evidence generator rather than as a replacement for semantic reasoning.

4.4 Semantic Retrieval Layer

The semantic retrieval stage addresses the lack of repository context. Code chunks are represented using CodeBERT embeddings and indexed in FAISS. The system retrieves semantically similar chunks rather than relying only on lexical matching.

The retrieval corpus is based on CodeSearchNet and selected public GitHub repositories. The prototype retrieval corpus contains eight chunks. This is sufficient for feasibility testing but should not be interpreted as a large-scale repository benchmark.

4.5 Retrieval-Augmented Prompt Construction

RAG connects retrieval with generation. For a submitted code fragment, the system constructs a contextual prompt from four main information sources: (i) the original source code; (ii) static-analysis findings; (iii) retrieved semantic context; and (iv) review instructions. This architecture differs from a direct "source code to LLM" workflow. Retrieved evidence provides additional context, while static findings provide deterministic signals. The LLM is used for explanation, reasoning, prioritization, and suggested remediation.

The prompt is constructed to preserve the distinction between evidence and generated reasoning. Static-analysis results are passed as explicit findings. Retrieved code is supplied as contextual evidence. The original code remains the primary object of review. A conceptual representation of the prompt is

$$P = C + S + R + I, \tag{1}$$

where C represents the submitted source code, S represents static-analysis findings, R represents retrieved semantic context, and I represents review instructions. The LLM then produces

$$O = f_{\text{CodeLlama}}(P), \tag{2}$$

where O is the generated review output.

4.6 CodeLlama Generation

CodeLlama is used as the code-specialized generation component. The model receives a structured prompt and produces review information. The proposed implementation uses a pretrained code-specialized LLM together with retrieval and prompt construction rather than relying on task-specific model fine-tuning.

4.7 Severity Normalization and Explainable Output

The project uses structured output parsing and deterministic severity normalization as implementation-level controls. The purpose is not to claim that the LLM has learned a universally correct severity scale. Instead, the framework maps generated findings into a consistent presentation format so that developers can compare issues within a review. The final review is structured into the following fields:

- issue title;
- issue description;
- severity;
- evidence;
- confidence; and

- suggested fix.

This output format is intended to make the review easier for developers to inspect and to provide a consistent presentation across programming languages.

5 Experiments and Results

5.1 Datasets and Technology Stack

The datasets are used at different stages rather than as one homogeneous training set ([Tab. 2]). CodeSearchNet supplies source-code material for semantic retrieval. CodeXGLUE provides benchmark resources for code understanding and review-related evaluation. Selected public GitHub repositories provide repository context for retrieval and testing.

Dataset/source	Role
CodeSearchNet	Similar-code repository and semantic retrieval
CodeXGLUE	Code-review and code-understanding evaluation resource
GitHub public repositories	Repository context, RAG retrieval, and testing

Table 2: Datasets and their roles.

The implementation uses Python as the primary programming language and Streamlit as the user interface. CodeLlama is the code-oriented LLM, CodeBERT is used for embeddings, and FAISS is used for vector similarity search. Pylint, PMD, and ESLint provide language-specific static analysis. [Tab. 3] lists the technology stack and the functional role of each component.

Component	Function
Python	Application implementation
Streamlit	User interface and review presentation
CodeLlama	Code review generation and explanation
CodeBERT	Semantic code representation
FAISS	Vector similarity search
Pylint	Python static analysis
PMD	Java static analysis
ESLint	JavaScript static analysis
RAG	Context retrieval and prompt augmentation

Table 3: Technology stack and functional roles.

5.2 Experimental Setup

The evaluation intentionally distinguishes system behavior from claims about universal code-review correctness. Precision, recall, F1-score, and accuracy are reported for the supplied prototype evaluation. The prototype evaluation uses a multilingual static-analysis dataset and a small retrieval corpus. The experimental design measures the performance of the system at multiple levels rather than relying on a single aggregate score. First, the static-analysis stage is evaluated separately for Python, Java, and JavaScript. Second, the semantic retrieval component is evaluated using rank-based retrieval metrics. Third, the final review output is evaluated for issue identification and structured review characteristics.

The experiments are intended as a feasibility evaluation of the integrated prototype. They should not be interpreted as a matched comparison against every published code-review model.

5.3 Evaluation Metrics

The evaluation is organized into five complementary dimensions: static-analysis performance, semantic retrieval quality, generated-review quality, human usefulness, and component contribution. This broader evaluation design follows recent code-review research, particularly work emphasizing contextual retrieval, human assessment, and ablation of retrieval and guidance components [Jaoua et al. 2025], [Meng et al. 2025], [Zhang et al. 2026].

5.3.1 Static-Analysis Metrics

For multilingual static-analysis results, the project reports precision, recall, F1-score, and accuracy. These metrics measure whether the deterministic analyzers correctly identify the evaluated issues.

5.3.2 Retrieval Metrics and Top-k Evaluation

Retrieval is evaluated using Recall@k and Mean Reciprocal Rank (MRR). The use of ranked retrieval measures is appropriate because RAG systems depend not only on whether a relevant item is retrieved, but also on where that item occurs in the ranked context [Meng et al. 2025], [Zhang et al. 2026]. The planned retrieval experiment evaluates multiple retrieval quantities, namely $k = 1, 2, 3, 5$, and 8, to study the effect of

increasing the amount of contextual evidence. Recall at rank k is defined as

$$\text{Recall@}k = \frac{\text{queries with a relevant item in top-}k}{\text{total queries}}. \tag{3}$$

Mean reciprocal rank is calculated as

$$\text{MRR} = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{\text{rank}_i}, \tag{4}$$

where rank_i is the position of the first relevant retrieved item for query i.

5.3.3 LLM-Based Review Quality

In addition to issue-level precision, recall, and F1-score, the project adopts five review-quality dimensions used in recent LLM-based code review research: readability, relevance, brevity, sufficiency, and operability [Zhang et al. 2026]. Each dimension is scored on a 1–5 scale, where higher values indicate better review quality.

- *Readability:* whether the review is clear and easy for a developer to understand;
- *Relevance:* whether the review directly addresses the submitted code and its specific issues;
- *Brevity:* whether the review communicates the issue without unnecessary or excessive explanation;
- *Sufficiency:* whether the review adequately covers the relevant issues and identifies their locations; and
- *Operability:* whether the recommendation provides practical guidance that a developer can act upon.

The existing prototype metrics are retained alongside these dimensions: precision, recall, F1-score, severity accuracy, evidence-grounding rate, and unsupported-finding rate. This prevents the qualitative review assessment from replacing the project's existing issue-detection evaluation.

5.3.4 Text-Generation Metrics

BLEU-4, ROUGE-L, and METEOR are included as supplementary automatic text-similarity measures. BLEU evaluates modified n-gram precision with a brevity penalty [Papineni et al. 2002]; ROUGE-L uses the longest common subsequence between generated and reference text [Lin 2004]; and METEOR incorporates unigram matching with stemming and synonym-aware alignment [Banerjee and Lavie 2005]. These metrics are useful for reproducible comparison with reference reviews, but they are not treated as the sole indicators of code-review quality because semantically equivalent review comments may use different wording [Zhang et al. 2026]. The project review

documentation also identifies CodeBLEU as a potential evaluation metric for code-related generation; however, no CodeBLEU numerical result is reported in the supplied comparative results, and therefore no CodeBLEU score is reported in this paper.

5.3.5 Human Evaluation

Because automated similarity metrics cannot fully determine whether a review is useful to a developer, the project also adopts the human evaluation categories used by LAURA [Zhang et al. 2026]. Each generated review is classified as Instrumental (I), Helpful (H), Uncertain (U), or Misleading (M). Instrumental reviews fully match the expected issues or provide equivalent valid solutions without obvious errors. Helpful reviews partially match the expected review and provide useful suggestions but may lack completeness or accuracy. Uncertain reviews do not overlap with the ground truth but provide potentially useful additional observations. Misleading reviews contradict the expected findings, contain irrelevant or incomprehensible content, or fail to provide a meaningful review. The following three metrics are calculated:

$$\text{I-Score} = \frac{I}{I+H+U+M} \times 100, \tag{5}$$

$$\text{IH-Score} = \frac{I+H}{I+H+U+M} \times 100, \tag{6}$$

$$\text{M-Score} = \frac{M}{I+H+U+M} \times 100. \tag{7}$$

Higher I-Score and IH-Score values are desirable, whereas a lower M-Score is desirable. Inter-rater agreement should also be reported when multiple human evaluators annotate the same reviews.

5.3.6 Ablation Studies

An ablation study is planned to quantify the contribution of the major components of the proposed hybrid architecture. This follows the evaluation strategy of recent retrieval-augmented code-review work, where contextual augmentation, review-exemplar retrieval, and guidance components are separately removed [Zhang et al. 2026]. The proposed configurations are:

1. CodeLlama only;
2. CodeLlama + static analysis;
3. CodeLlama + RAG;
4. static analysis + RAG + CodeLlama; and
5. the full hybrid system with static analysis, CodeBERT, FAISS, RAG, CodeLlama, and structured output.

The ablation experiments will compare issue-level precision, recall, F1-score, review-

quality scores, retrieval quality, severity accuracy, and evidence grounding. The purpose is to determine whether each architectural component contributes measurable value rather than assuming that the complete pipeline is automatically superior.

5.4 Results

5.4.1 Static-Analysis Performance

[Tab. 4] and [Fig. 3] report the measured performance for each programming language. Python obtained precision of 0.50, recall of 1.00, F1-score of 0.67, and accuracy of 0.50. Java obtained precision of 0.25, recall of 1.00, F1-score of 0.40, and accuracy of 0.25. JavaScript obtained 1.00 across all four reported metrics. The overall prototype values were precision 0.56, recall 1.00, F1-score 0.71, and accuracy 0.56.

Language	Precision	Recall	F1	Accuracy
Python	0.50	1.00	0.67	0.50
Java	0.25	1.00	0.40	0.25
JavaScript	1.00	1.00	1.00	1.00
Overall	0.56	1.00	0.71	0.56

Table 4: Static-analysis performance by programming language.

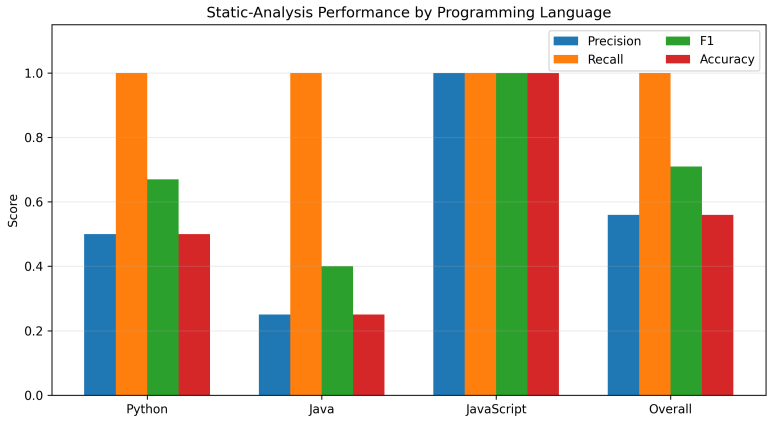


Figure 3: Static-analysis performance of the proposed project by programming language.

The language-wise differences are important. The perfect recall reported for the prototype indicates that all evaluated static findings were captured by the evaluation procedure. The lower precision in Python and Java indicates that additional findings were produced. These results support the need for contextual reasoning and evidence vali-

dation rather than interpreting every static or generated finding as an equally reliable defect.

5.4.2 Retrieval Performance

The retrieval component achieved Recall@1 of 0.67 and Recall@2 of 0.67 ([Tab. 5], [Fig. 4]). Recall@3 reached 1.00, meaning that every evaluated retrieval case had a relevant result within the top three positions. The reported MRR of 0.78 indicates that relevant results tended to occur relatively high in the ranked list. The results demonstrate feasibility on the prototype retrieval corpus. However, the corpus is small, and retrieval performance may change substantially when the system is evaluated against larger repositories containing different programming styles, project structures, and domain-specific code.

Metric	Result
Recall@1	0.67
Recall@2	0.67
Recall@3	1.00
MRR	0.78

Table 5: Semantic retrieval results.

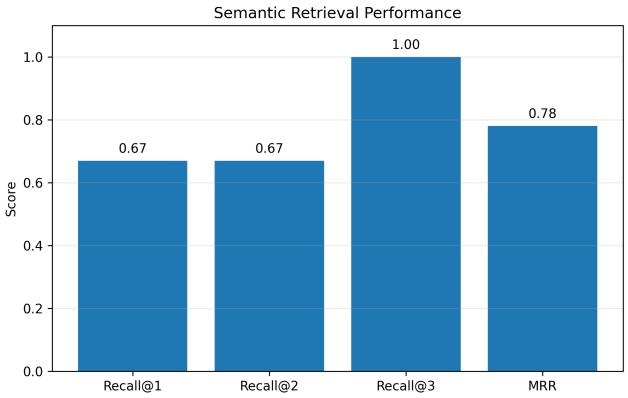


Figure 4: Semantic retrieval performance of the CodeBERT–FAISS component.

5.4.3 Review Quality

[Tab. 6] summarizes the reported review-quality results. Precision is 1.00, recall is 0.67, and F1-score is 0.80. Severity accuracy and evidence-grounding rate are both reported

as 1.00, while the unsupported-finding rate is 0.00. These values describe the evaluated prototype review behavior and should not be interpreted as universal LLM accuracy. In particular, the project's evaluation is based on a small selected set and does not replace human assessment of the usefulness and correctness of generated review comments.

Metric	Result
Precision	1.00
Recall	0.67
F1-score	0.80
Severity accuracy	1.00
Evidence grounding rate	1.00
Unsupported finding rate	0.00

Table 6: Review quality evaluation.

5.5 Comparative Analysis

The comparative analysis separates qualitative system capabilities from numerical evaluation. This distinction is necessary because the reviewed approaches use different datasets, tasks, labels, and scoring procedures.

5.5.1 Qualitative Comparison

[Tab. 7] compares the documented scope of selected approaches; "NS" means that the supplied comparison material did not provide sufficient evidence to mark a feature as implemented, and [Fig. 5] visualizes the coverage. The proposed project combines six functional areas: language-specific static analysis; semantic retrieval; review generation; structured and explainable output; severity mapping; and multilingual support. This integrated scope is the principal system-level contribution rather than a claim of a new standalone learning algorithm.

5.5.2 Numerical Comparison

[Tab. 8] reports the numerical values supplied in the comparative-analysis document. The proposed project records higher recall than the reported comparison values, but the proposed precision and accuracy are not directly comparable because the tasks differ. The CodeReviewer and baseline values concern code-change quality estimation, whereas the proposed project's values concern static-analysis findings on a small multilingual prototype dataset. Therefore, [Fig. 6] is a reported-metrics reference rather than

Approach	Static	RAG	Review gen.	Structured output	Severity	Multi-lang.	Main difference
LLM + static analyzers (2025)	Yes	Partial	Yes	NS	NS	NS	Combines analyzer signals and LLM generation.
RepoAudit-style system	NS	Yes	Yes	Partial	NS	NS	Repository-level auditing and validation are central.
Fine-tuned LLM review	NS	NS	Yes	Partial	NS	NS	Requires task-specific review data and training.
CodeReviewQA	NC	NC	Eval.	Eval.	NC	NC	Primarily a review-comprehension benchmark.
CodeAgent (2024)	Partial	NC	Yes	Yes	NS	NS	Multi-agent coordination increases workflow complexity.
Proposed project	Yes	Yes	Yes	Yes	Yes	Yes	Integrates all listed functions in one application.

Table 7: Qualitative comparison of code-review approaches.

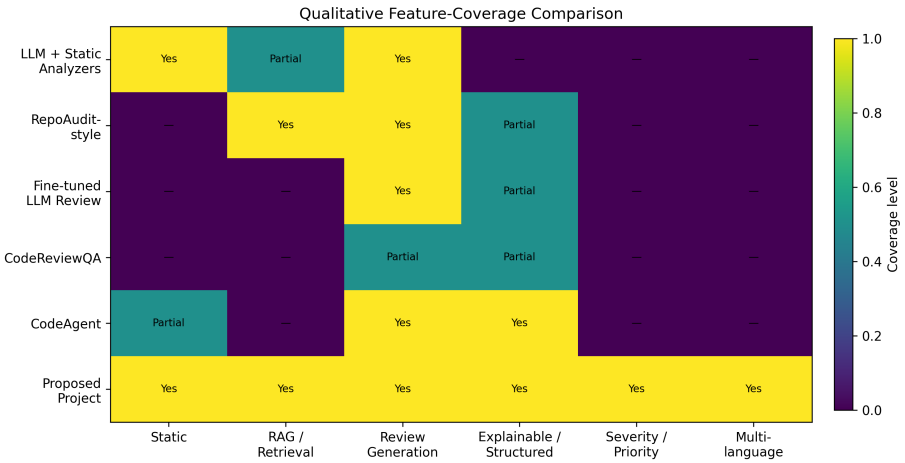


Figure 5: Qualitative feature-coverage comparison of selected approaches and the proposed project.

evidence of universal superiority.

System	Task	Precision	Recall	F1	Accuracy
Proposed project	Small multilingual static-analysis dataset	0.560	1.000	0.710	0.560
CodeReviewer	Code-change quality estimation	0.786	0.656	0.715	0.739
T5 baseline	Code-change quality estimation	0.708	0.572	0.633	0.668
CodeT5 baseline	Code-change quality estimation	0.704	0.590	0.642	0.671
Transformer baseline	Code-change quality estimation	0.745	0.461	0.569	0.652

Table 8: Reported metrics across different tasks and datasets.

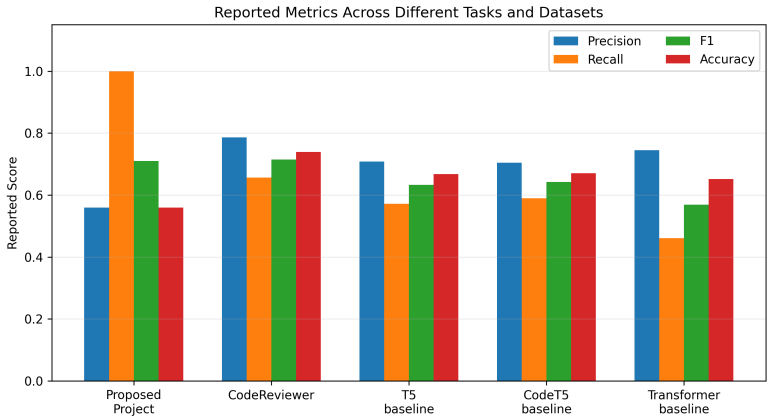


Figure 6: Reported metric comparison across systems evaluated under different tasks and datasets.

5.6 Discussion

The experimental results demonstrate that the proposed pipeline is operational across its principal stages. The static-analysis results confirm that the language-specific analysis layer can be applied to three programming languages in a common application. The retrieval results demonstrate that semantic search can supply relevant context for the evaluated prototype cases. The review-quality results indicate that the structured output stage can produce consistent severity and evidence fields under the selected evaluation procedure.

The most important architectural advantage is the separation of evidence generation from reasoning. Static analyzers provide deterministic signals, retrieval supplies contextual information, and the LLM performs explanation and recommendation. This separation makes it possible to inspect intermediate evidence rather than treating the LLM response as an opaque decision.

The retrieval stage also introduces an important scalability consideration. As repository size increases, the quality of the retrieved context becomes increasingly important. Poorly retrieved code can introduce irrelevant information into the prompt and potentially affect the generated review. Consequently, future work should investigate retrieval thresholds, chunking strategies, metadata filtering, and larger repository corpora.

The severity-normalization layer provides another practical benefit. LLMs can use different terminology or severity scales in different responses, and mapping results to a consistent set of severity categories makes review reports easier to interpret. However, severity accuracy in the current evaluation should not be treated as proof that the system's severity labels correspond perfectly to developer judgments; human evaluation is required for that claim. The Streamlit interface additionally provides a single workflow for submitting code and viewing the structured review, which supports the project's stated objective of reducing manual review effort and providing developer-friendly explanations.

5.7 Creativity and Feasibility

The framework simulates an expert-review workflow in which deterministic tools identify candidate issues and a code-specialized LLM explains them using contextual evidence. The modular architecture allows individual components to be replaced or expanded without redesigning the complete application; for example, additional static analyzers or programming languages can be integrated in future versions.

Technically, the system uses established open-source models, libraries, and analysis tools. CodeBERT and FAISS provide an established route for semantic retrieval, CodeLlama supplies code-oriented language generation, and Pylint, PMD, and ESLint are compatible with language-specific static analysis, so these components can be integrated within a Python application. Economically, the framework relies primarily on open-source technologies and does not require expensive model training. Operationally, the Streamlit interface provides a relatively simple interaction model for developers, and the modular architecture supports future integration with developer workflows, IDEs, and CI/CD systems.

The implementation proceeds from problem and literature analysis to architecture design, integration of the static-analysis and retrieval components, LLM prompt construction, interface development, testing, and evaluation. This phased implementation permits individual components to be evaluated before end-to-end integration.

5.8 Limitations and Future Work

The current evaluation has several limitations. First, the prototype dataset is small. Second, the retrieval corpus is limited, so the reported retrieval metrics should not be generalized to large repositories. Third, the numerical comparison uses different tasks

and datasets and therefore cannot establish a direct ranking against published systems. Fourth, human evaluation of the generated comments has not been established in the supplied experimental results. The absence of matched benchmark experiments is particularly important. A stronger evaluation would use the same dataset, train/test split, label definition, evaluation metrics, and preprocessing procedure for the proposed system and selected baselines, which would allow a statistically meaningful comparison. Future work should therefore include:

- larger and more diverse repositories;
- human evaluation of review usefulness;
- ablation studies that separately remove static analysis, retrieval, RAG, and structured output;
- latency and memory measurements;
- matched benchmark experiments;
- additional programming languages; and
- IDE and CI/CD integrations.

A further direction is improved retrieval: the current system can be extended with repository metadata, file-path awareness, symbol-level indexing, dependency relationships, and historical review information, which could help retrieve context that is more closely related to the code under review. Another direction is evaluation of generated fixes. The current results emphasize review findings and structured explanations; future experiments could assess whether suggested fixes compile, pass tests, and preserve intended functionality, providing a stronger measure of practical usefulness than text similarity alone.

6 Conclusion

This paper presented an AI Copilot for context-aware automated code review that combines language-specific static analysis, CodeBERT semantic retrieval, FAISS vector search, Retrieval-Augmented Generation, CodeLlama, and structured explainable reporting. The system supports Python, Java, and JavaScript and integrates deterministic analysis with contextual LLM reasoning. The reported prototype results show overall static-analysis precision of 0.56, recall of 1.00, F1-score of 0.71, and accuracy of 0.56. The retrieval component achieved Recall@3 of 1.00 and MRR of 0.78. Review-quality evaluation reported precision of 1.00, recall of 0.67, F1-score of 0.80, severity accuracy of 1.00, evidence-grounding rate of 1.00, and unsupported-finding rate of 0.00. These results demonstrate the feasibility of the integrated architecture on the selected prototype evaluation set.

The primary contribution is the combination of complementary mechanisms into one developer-oriented workflow: static analyzers supply deterministic evidence; CodeBERT and FAISS supply semantic context; RAG organizes the evidence; and CodeLlama provides explanation and suggested remediation. The results should nevertheless be interpreted conservatively. The prototype dataset and retrieval corpus are limited,

and the reported comparison with published systems uses different tasks and datasets. Larger matched evaluations, human assessment, ablation studies, and performance profiling are required before broader claims can be made. With these extensions, the framework can evolve toward a more comprehensive context-aware code-review assistant for practical software-development environments.

References

[Rozière et al. 2023] Rozière, B. et al.: "Code Llama: Open Foundation Models for Code"; arXiv preprint arXiv:2308.12950 (2023).

[Feng et al. 2020] Feng, Z. et al.: "CodeBERT: A Pre-Trained Model for Programming and Natural Languages"; Findings of EMNLP (2020), 1536–1547.

[Lu et al. 2021] Lu, S. et al.: "CodeXGLUE: A Machine Learning Benchmark Dataset for Code Understanding and Generation"; Proc. NeurIPS Datasets and Benchmarks (2021).

[Husain et al. 2019] Husain, H. et al.: "CodeSearchNet Challenge: Evaluating the State of Semantic Code Search"; Proc. 2020 Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (2019).

[Johnson et al. 2021] Johnson, J., Douze, M., Jégou, H.: "Billion-Scale Similarity Search with GPUs"; IEEE Trans. Big Data, 7, 3 (2021), 535–547.

[Lewis et al. 2020] Lewis, P. et al.: "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks"; Advances in Neural Information Processing Systems, 33 (2020).

[Rasheed et al. 2024] Rasheed, Z., Sami, M. A., Waseem, M., Kemell, K.-K., Wang, X., Nguyen, A., Systä, K., Abrahamsson, P.: "AI-Powered Code Review with LLMs: Early Results"; arXiv preprint arXiv:2404.18496 (2024).

[Tang et al. 2024] Tang, X. et al.: "CodeAgent: Autonomous Communicative Agents for Code Review"; Proc. 2024 Conf. Empirical Methods in Natural Language Processing (2024), 11279–11313.

[Guo et al. 2025] Guo, J., Wang, C., Xu, X., Su, Z., Zhang, X.: "RepoAudit: An Autonomous LLM-Agent for Repository-Level Code Auditing"; Proc. 42nd Int. Conf. Machine Learning, PMLR, 267 (2025), 21083–21100.

[Yu et al. 2025] Yu, Y., Rong, G., Shen, H., Zhang, H., Shao, D., Wang, M., Wei, Z., Xu, Y., Wang, J.: "Fine-Tuning Large Language Models to Improve Accuracy and Comprehensibility of Automated Code Review"; ACM Trans. Softw. Eng. Methodol., 34, 1 (2025), 1–26.

[Lin et al. 2025] Lin, H. Y., Liu, C., Gao, H., Thongtanunam, P., Treude, C.: "CodeReviewQA: The Code Review Comprehension Assessment for Large Language Models"; Findings of ACL 2025 (2025), 9138–9166.

[Jaoua et al. 2025] Jaoua, I., Ben Sghaier, O., Sahraoui, H.: "Combining Large Language Models with Static Analyzers for Code Review Generation"; Proc. 2025 IEEE/ACM 22nd Int. Conf. Mining Software Repositories (2025), doi: 10.1109/MSR66628.2025.00038.

[Meng et al. 2025] Meng, Q., Zhang, X., Ren, Z., Visser, J.: "Code Review Automation using Retrieval Augmented Generation"; arXiv preprint arXiv:2511.05302 (2025).

[Zhang et al. 2026] Zhang, Y., Zhang, Y., Sun, Z., Jiang, Y., Liu, H.: “LAURA: LLM AUgmentation Framework for Code Review GenerAtion”; arXiv preprint arXiv:2512.01356v2 (2026).

[Papineni et al. 2002] Papineni, K., Roukos, S., Ward, T., Zhu, W.-J.: “BLEU: A Method for Automatic Evaluation of Machine Translation”; Proc. 40th Annual Meeting of the Association for Computational Linguistics (2002), 311–318.

[Lin 2004] Lin, C.-Y.: “ROUGE: A Package for Automatic Evaluation of Summaries”; Text Summarization Branches Out (2004), 74–81.

[Banerjee and Lavie 2005] Banerjee, S., Lavie, A.: “METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments”; Proc. ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization (2005), 65–72.